Complete Lab Guide: Ecosystem Optimization with Evolutionary Algorithms

Part 1: The Big Picture - What This Lab Is About

This lab tackles a real-world simulation problem: creating a stable ecosystem where three species (carrots, rabbits, and eagles) coexist without any going extinct.

The Challenge: You need to find the "perfect" initial conditions—grid size, number of animals, obstacle placement—that keep all species alive for 100 simulation steps.

The Approach: You'll use two different evolutionary algorithms (ES and GA) to automatically search for these optimal conditions, then compare which works better.

Why this matters: This mirrors real problems like wildlife management, resource allocation, or tuning complex systems where you have many interacting parameters.

Part 2: Understanding the Ecosystem Simulation

The World

Grid-Based Environment:

- The world is an L × L grid (think chess board, but bigger)
- L can range from 25-60 cells (depends on exercise)
- Each cell can contain: empty space, obstacles, carrots, rabbits, or eagles

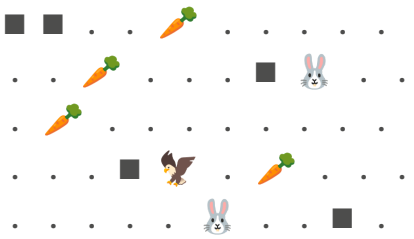
Obstacles:

- Black squares that block movement
- A fraction p_obs of the grid is obstacles (e.g., 7.5% of cells)
- Critical function: Obstacles block eagle line-of-sight—rabbits can hide behind them!

Visualization:

```
text
```

```
Grid (L=10 example):
■ = obstacle
🥕 = carrot
🐰 = rabbit
🦅 = eagle
```



The Species

Carrots (Producers):

- Stationary resources
- Reproduce asexually with fixed probability (r_c = 0.05 per step)
- Spawn into adjacent empty cells
- Role: Food for rabbits

Rabbits (Herbivores):

- Move up to 2 cells per step
- Can see 3 cells in any direction
- Behavior:
    - If carrot detected → move toward it
    - If carrot in same cell → eat it (gain +6 energy)
    - Otherwise → random walk
- Survival needs: Must eat regularly or starve (energy decreases by 1 each step)

Eagles (Predators):

- Move up to 3 cells per step (faster than rabbits!)
- Can see 5 cells in any direction
- Special ability: Line-of-sight detection with obstacle occlusion
    - Eagles trace a ray from their position to each rabbit
    - If an obstacle blocks the ray, that rabbit is invisible
- Behavior:
    - If rabbit visible → chase it

- If rabbit in same cell → catch it (gain +8 energy)
- Otherwise → random walk

Life Cycle Mechanics

Energy System:

- Every animal has an energy value
- Energy decreases by 1 each step (metabolism)
- Energy = 0 → death (starvation)
- Eating replenishes energy (capped at maximum)

Age and Maturity:

- Animals start as juveniles
- After 5 steps → become adults
- Only adults can reproduce

Reproduction:

- Requires: adult male + adult female in same cell
- Both parents pay energy cost (3 each)
- Offspring spawns in random adjacent empty cell (Moore neighborhood = 8 surrounding cells)
- If no empty adjacent cell → reproduction fails
- Offspring inherits species, gets random sex, starts with initial energy

Sex Ratios (ρ):

- rho_r (ρ_r): fraction of rabbits that are female (e.g., 0.50 = 50% female)
- rho_e (ρ_e): fraction of eagles that are female
- Why it matters: Too few females or males → no reproduction → extinction!

Update Order (Crucial for Understanding):

Each simulation step follows this exact sequence:

1. Record populations (for tracking)
2. Carrot reproduction (asexual spawning)
3. Rabbit movement (toward carrots or random)
4. Eagle movement (toward visible rabbits or random)
5. Interactions (rabbits eat carrots, eagles catch rabbits)
6. Reproduction (if conditions met)
7. Age and decay (animals age 1 step, energy decreases 1)

8. Cull dead (remove energy=0 animals)

Why order matters: If eagles moved before rabbits, rabbits couldn't escape. If reproduction happened before eating, animals might reproduce then die immediately!

Part 3: The Optimization Problem

Decision Variables (What We're Optimizing)

You control 7 parameters:

Variable	**Meaning**	**Type**	**Range**	**Why It Matters**
L	Grid size	Integer	[25-60]	Bigger = more space, but harder to find food/mates
p_obs	Obstacle fraction	Real	[0.05-0.10]	More obstacles = more hiding spots for rabbits, but blocks movement
N_e0	Initial eagles	Integer	[2-8]	Too many = rabbits die out; too few = overpopulation
N_r0	Initial rabbits	Integer	[20-200]	Must sustain eagle population
N_c0	Initial carrots	Integer	[300-600]	Must sustain rabbit population

ρ_e	Eagle female ratio	Real	[0.40-0.60]	Need balanced sexes for reproduction
ρ_r	Rabbit female ratio	Real	[0.40-0.60]	Need balanced sexes for reproduction

Representation: These are encoded as a vector θ = [L, p_obs, N_e0, N_r0, N_c0, ρ_e, ρ_r]

The Objective: Stability

Hard Constraint:

- NO species may go extinct before 100 steps
- If any species hits 0 population → solution is heavily penalized

Soft Objectives (for solutions that survive):

1. Maximize minimum population: Want healthy populations (not just barely alive)
2. Minimize volatility: Stable populations (not wild oscillations)
3. Minimize overcrowding: Don't pack too many agents (realistic dynamics)

Fitness Function (How Solutions Are Scored)

Per-Seed Evaluation:

```
python
```

```
If any species goes extinct before step 100:
    fitness = -50 × (100 - time_of_extinction)
    # Heavy penalty: earlier extinction = worse score
    # Example: extinction at step 70 → fitness = -50 × 30 = -1500

Else (all survived 100 steps):
    min_pop = minimum population of any species during run
    volatility = sum of mean absolute changes per species
    overcrowding = penalty if density > 80% of free cells
```

```
    fitness = 1.0 × min_pop - 1.0 × volatility - 1.0 ×
overcrowding
    # Example: min_pop=50, vol=20, over=5 → fitness = 50-20-5 = 25
```

Robust Aggregation:

- Run the same parameters with S=5 different random seeds
- Take the median fitness across seeds
- Why median? Robust to noise—one unlucky/lucky run doesn't dominate

Stochasticity Sources:

- Random obstacle placement
- Random initial agent positions
- Random movement tie-breaking
- Random reproduction spawning locations

Part 4: Evolution Strategy (ES) Implementation - Exercise 1

What Is ES? (Quick Refresher)

Evolution Strategy is a real-valued continuous optimizer that:

- Maintains a population of candidate solutions
- Uses mutation as the primary operator (adding Gaussian noise)
- Selects the best offspring to update the population mean
- Works in (μ, λ) mode: μ parents create λ offspring, select best μ from offspring only

Your TODO Tasks for ES

Task 1: Implement `clamp(x, lo, hi)`

Purpose: Ensure a value stays within bounds.

```
python
```

```python
def clamp(x: float, lo: float, hi: float) -> float:
    """
    Force x to be between lo and hi.
```

```
    Examples:
        clamp(15, 10, 20) → 15 (already in range)
        clamp(5, 10, 20) → 10 (too low, bump to lower bound)
        clamp(25, 10, 20) → 20 (too high, cap at upper bound)
    """
    return min(max(x, lo), hi)
    # Or use: return float(np.clip(x, lo, hi))
```

Why needed: Mutation might push parameters out of valid ranges. Clamping repairs them.

Task 2: Implement `decode(theta_real)`

Purpose: Convert the real-valued ES vector into actual simulation parameters.

python

```
def decode(theta_real: np.ndarray) -> DecodedParams:
    """
    Transform real-valued vector into bounded, typed parameters.

    Input: theta_real = [L_raw, pobs_raw, Ne0_raw, Nr0_raw,
Nc0_raw, rhoe_raw, rhor_raw]
    Output: DecodedParams with clamped and rounded values
    """
    # Clamp all values to their respective bounds
    L_clamped = clamp(theta_real[0], MIN_L, MAX_L)
    pobs_clamped = clamp(theta_real[1], MIN_POBS, MAX_POBS)
    Ne0_clamped = clamp(theta_real[2], MIN_EAGLES, MAX_EAGLES)
    Nr0_clamped = clamp(theta_real[3], MIN_RABBITS, MAX_RABBITS)
    Nc0_clamped = clamp(theta_real[4], MIN_CARROTS, MAX_CARROTS)
    rhoe_clamped = clamp(theta_real[5], MIN_RHO, MAX_RHO)
    rhor_clamped = clamp(theta_real[6], MIN_RHO, MAX_RHO)

    # Round integer parameters
    L = int(round(L_clamped))
```

```
    Ne0 = int(round(Ne0_clamped))
    Nr0 = int(round(Nr0_clamped))
    Nc0 = int(round(Nc0_clamped))

    # Keep real parameters as floats
    pobs = float(pobs_clamped)
    rhoe = float(rhoe_clamped)
    rhor = float(rhor_clamped)

    return DecodedParams(
        L=L, p_obs=pobs,
        N_e0=Ne0, N_r0=Nr0, N_c0=Nc0,
        rho_e=rhoe, rho_r=rhor
    )
```

Key insights:

- Integer parameters (L, N_e0, N_r0, N_c0) must be rounded to whole numbers
- Real parameters (p_obs, rho_e, rho_r) stay as floats
- All values clamped first, then rounded (prevents invalid integers)

Task 3: Implement `fitness(theta_real, base_seed, S, steps)`

Purpose: Evaluate a solution robustly across multiple random seeds.

python

```
def fitness(theta_real: np.ndarray, base_seed: int = 12345, S:
int = 5, steps: int = SIM_STEPS) -> float:
    """
    Robust fitness: median score across S different seeds.

    Process:
    1. Decode theta into parameters
    2. Run S simulations with different seeds
    3. Compute fitness for each run
    4. Return median fitness
    """
```

```
    # Decode into simulation parameters
    params = decode(theta_real)

    # Collect fitness from S independent runs
    fitness_values = []
    for i in range(S):
        seed = base_seed + i * 1000  # Generate different seeds
        fit = fitness_one_seed(params, seed, steps)  # Provided
function
        fitness_values.append(fit)

    # Return median (robust to outliers)
    return float(np.median(fitness_values))
```

Why median?: If 4 runs succeed (fitness ~30) but 1 fails catastrophically (fitness -1500), mean would be heavily dragged down. Median stays representative.

Task 4: Implement `run_es(...)`

Purpose: The main Evolution Strategy loop.

python

```
def run_es(
    generations: int = 80,
    mu: int = 10,           # Number of parents
    lam: int = 40,          # Number of offspring
    seed: int = 7,
    S: int = 5,             # Seeds per evaluation
    steps: int = SIM_STEPS,
    verbose_every: int = 1,
):
    """
    (μ, λ) Evolution Strategy.

    Algorithm:
    1. Initialize mean vector
```

```
2. For each generation:
   a. Sample λ offspring via Gaussian mutation
   b. Evaluate each offspring
   c. Select top μ offspring
   d. Update mean as average of top μ
   e. Log progress
3. Return best solution found
"""
rng = np.random.default_rng(seed)

# Initialize mean vector (center of search space)
theta_mean = np.array([
    (MIN_L + MAX_L) / 2,
    (MIN_POBS + MAX_POBS) / 2,
    (MIN_EAGLES + MAX_EAGLES) / 2,
    (MIN_RABBITS + MAX_RABBITS) / 2,
    (MIN_CARROTS + MAX_CARROTS) / 2,
    (MIN_RHO + MAX_RHO) / 2,
    (MIN_RHO + MAX_RHO) / 2,
], dtype=float)

# Initial step sizes (standard deviations for mutation)
sigma = np.array([
    (MAX_L - MIN_L) * 0.15,           # ~15% of range
    (MAX_POBS - MIN_POBS) * 0.15,
    (MAX_EAGLES - MIN_EAGLES) * 0.15,
    (MAX_RABBITS - MIN_RABBITS) * 0.15,
    (MAX_CARROTS - MIN_CARROTS) * 0.15,
    (MAX_RHO - MIN_RHO) * 0.15,
    (MAX_RHO - MIN_RHO) * 0.15,
], dtype=float)

best_theta = theta_mean.copy()
best_fitness = -np.inf
history = []
```

```
    for gen in range(generations):
        # Generate λ offspring via Gaussian mutation
        offspring = []
        offspring_fitness = []

        for _ in range(lam):
            # Add Gaussian noise to mean
            noise = rng.normal(0, sigma)
            child = theta_mean + noise
            offspring.append(child)

        # Evaluate all offspring
        base_seed = 10000 + gen * 1000
        for child in offspring:
            fit = fitness(child, base_seed=base_seed, S=S,
steps=steps)
            offspring_fitness.append(fit)

        # Select top μ offspring
        indices = np.argsort(offspring_fitness)[-mu:]  # Best μ
indices
        top_offspring = [offspring[i] for i in indices]
        top_fitness = [offspring_fitness[i] for i in indices]

        # Update mean as average of top μ
        theta_mean = np.mean(top_offspring, axis=0)

        # Track best solution ever found
        current_best_idx = indices[-1]
        current_best_fit = offspring_fitness[current_best_idx]
        if current_best_fit > best_fitness:
            best_fitness = current_best_fit
            best_theta = offspring[current_best_idx].copy()
```

```
        # Log progress
        if (gen + 1) % verbose_every == 0:
            median_fit = np.median(offspring_fitness)
            best_decoded = decode(best_theta)
            print(f"Gen {gen+1}/{generations} | "
                  f"Best: {best_fitness:.2f} | "
                  f"Median: {median_fit:.2f} | "
                  f"Decoded best: {best_decoded}")

        history.append({
            'gen': gen,
            'best': current_best_fit,
            'median': np.median(offspring_fitness),
        })

    return best_theta, best_fitness, history
```

Key ES Concepts Illustrated:

- (μ, λ) selection: Only offspring compete; parents don't survive
- Gaussian mutation: `theta + N(0, σ)` creates variations
- Mean update: Center of search shifts toward good regions
- No crossover: ES is mutation-driven, unlike GA

Part 5: Genetic Algorithm (GA) Implementation - Exercise 2

What Makes GA Different from ES?

Aspect	**ES**	**GA**
Primary operator	Mutation	Crossover

Representation	Real-valued vectors	Mixed (int + real)
Selection	Rank top μ	Tournament or roulette
Recombination	None (or intermediate)	Core mechanism
Best for	Continuous optimization	Discrete/mixed problems

Your TODO Tasks for GA

Task 1: Implement `tournament_select(pop, stats, rng, k=3)`

Purpose: Select one parent using tournament selection.

python

```python
def tournament_select(pop: List[np.ndarray], stats:
List[Dict[str, float]],
                      rng: np.random.Generator, k: int = 3) ->
np.ndarray:
    """
    Tournament selection: pick k random individuals, return the
best.

    Process:
    1. Randomly sample k indices from population
    2. Compare their stats using compare_stats(...)
    3. Return the genome of the best
    """
    # Sample k random indices (with replacement is fine)
    indices = rng.choice(len(pop), size=k, replace=True)
```

```
    # Find the best among them
    best_idx = indices[0]
    for idx in indices[1:]:
        # compare_stats(a, b) returns:
        #   1 if a is better
        #  -1 if b is better
        #   0 if equal
        if compare_stats(stats[idx], stats[best_idx]) > 0:
            best_idx = idx

    return pop[best_idx]
```

Why tournament selection?:

- Simple to implement
- k controls selection pressure directly (k=2 is mild, k=7 is aggressive)
- Doesn't require fitness scaling
- Feasibility-first: `compare_stats` always prefers feasible over infeasible solutions

Task 2: Implement `mutate(genome, rng, ...)`

Purpose: Introduce random variation to maintain diversity.

python

```
def mutate(genome: np.ndarray, rng: np.random.Generator,
           p_mut_gene: float = 0.25,
           int_step: Optional[Dict[int, int]] = None,
           real_sigma: Optional[Dict[int, float]] = None) ->
np.ndarray:
    """
    Per-gene mutation with different strategies for integer vs
real genes.

    Integer genes: add random integer step
    Real genes: add Gaussian noise
    """
```

```
mutant = genome.copy()

# Default mutation parameters if not provided
if int_step is None:
    int_step = {
        0: 5,   # L: ±5 cells
        2: 1,   # N_e0: ±1 eagle
        3: 10,  # N_r0: ±10 rabbits
        4: 30,  # N_c0: ±30 carrots
    }

if real_sigma is None:
    real_sigma = {
        1: 0.01,  # p_obs: ±0.01
        5: 0.05,  # rho_e: ±0.05
        6: 0.05,  # rho_r: ±0.05
    }

for i in range(len(mutant)):
    # Each gene has p_mut_gene probability of mutating
    if rng.random() < p_mut_gene:
        if i in INT_IDXS:  # Integer-like gene
            step = int_step.get(i, 1)
            delta = rng.integers(-step, step + 1)
            mutant[i] += delta
        elif i in REAL_IDXS:  # Real-valued gene
            sigma = real_sigma.get(i, 0.01)
            noise = rng.normal(0, sigma)
            mutant[i] += noise

# Repair: clamp to bounds
repair_inplace(mutant)  # Provided function

return mutant
```

Key differences from ES mutation:

- Selective: Only some genes mutate each time (controlled by p_mut_gene)
- Type-aware: Different strategies for integers vs reals
- Discrete for integers: Can't add 0.7 eagles—use integer steps!

Task 3: Implement `crossover(parent1, parent2, rng, ...)`

Purpose: Combine two parents to create offspring.

python

```
def crossover(parent1: np.ndarray, parent2: np.ndarray, rng:
np.random.Generator,
              p_swap_int: float = 0.5, alpha_blend: float =
0.25) -> np.ndarray:
    """
    Mixed-type crossover:
    - Integer genes: uniform swap
    - Real genes: BLX-α blend
    """
    child = np.empty(DIM, dtype=float)

    for i in range(DIM):
        if i in INT_IDXS:  # Integer gene
            # Uniform swap: randomly pick from either parent
            if rng.random() < p_swap_int:
                child[i] = parent1[i]
            else:
                child[i] = parent2[i]

        elif i in REAL_IDXS:  # Real gene
            # BLX-α (Blend crossover)
            # Creates value between parents, with α% extension
beyond range
            v1, v2 = parent1[i], parent2[i]
            vmin, vmax = min(v1, v2), max(v1, v2)
            range_val = vmax - vmin
```

```
            # Extend range by α on both sides
            lower = vmin - alpha_blend * range_val
            upper = vmax + alpha_blend * range_val

            # Sample uniformly in extended range
            child[i] = rng.uniform(lower, upper)

    # Repair bounds
    repair_inplace(child)

    return child
```

Crossover Intuition:

Integer uniform swap:

```
text
```

```
Parent 1: [L=30, p_obs=0.08, N_e0=4, ...]
Parent 2: [L=50, p_obs=0.06, N_e0=6, ...]

For N_e0 (integer):
- Coin flip: heads→take from P1 (4), tails→take from P2 (6)
- Child gets either 4 or 6
```

BLX-α for real values:

```
text
```

```
Parent 1: rho_e = 0.45
Parent 2: rho_e = 0.55
Range = 0.55 - 0.45 = 0.10

With α=0.25:
Extended range = [0.45 - 0.25×0.10, 0.55 + 0.25×0.10]
               = [0.425, 0.575]
```

```
Child gets random value in [0.425, 0.575]
```

Why BLX-α?: Allows "outside the box" values—child can exceed parent range slightly, enabling exploration beyond current population bounds.

Task 4: Implement `run_ga(...)`

Purpose: The main Genetic Algorithm loop.

python

```python
def run_ga(
    generations: int = 40,
    pop_size: int = 60,
    elite_frac: float = 0.10,
    tournament_k: int = 3,
    crossover_rate: float = 0.85,
    mutation_rate: float = 0.25,
    S: int = S_SEEDS,
    steps: int = SIM_STEPS,
    seed: int = 7,
    reevaluate_elites: bool = True,
    verbose_every: int = 1,
):
    """
    Genetic Algorithm with feasibility-first selection.
    """
    rng = np.random.default_rng(seed)

    # Initialize population
    pop = [random_genome(rng) for _ in range(pop_size)]

    # Track global best
    best_genome = None
    best_stat = {"feasible": 0.0, "score": -np.inf}

    n_elite = int(elite_frac * pop_size)
```

```
    for gen in range(generations):
        # Generate evaluation seeds for this generation
        eval_seeds = seed_set(base_seed=10000 + gen * 1000, S=S)

        # Evaluate entire population
        stats = [robust_eval(g, eval_seeds, steps=steps) for g
in pop]

        # Sort by (feasible desc, score desc)
        sorted_indices = sorted(
            range(pop_size),
            key=lambda i: (stats[i]["feasible"],
stats[i]["score"]),
            reverse=True
        )
        pop = [pop[i] for i in sorted_indices]
        stats = [stats[i] for i in sorted_indices]

        # Update global best
        if compare_stats(stats[0], best_stat) > 0:
            best_genome = pop[0].copy()
            best_stat = stats[0].copy()

        # Log progress
        if (gen + 1) % verbose_every == 0:
            feas_rate = np.mean([s["feasible"] for s in stats])
            median_score = np.median([s["score"] for s in
stats])
            print(f"Gen {gen+1}/{generations} | "
                  f"Feas: {feas_rate:.2%} | "
                  f"Best: {stats[0]['score']:.2f} | "
                  f"Median: {median_score:.2f} | "
                  f"Decoded: {decode(pop[0])}")
```

```
        # Create next generation
        next_pop = []

        # Elitism: copy top individuals
        elites = [g.copy() for g in pop[:n_elite]]

        # Optionally re-evaluate elites on fresh seeds
        if reevaluate_elites:
            fresh_seeds = seed_set(base_seed=20000 + gen * 1000,
S=S)
            elite_stats = [robust_eval(g, fresh_seeds,
steps=steps) for g in elites]
            next_pop.extend(elites)
        else:
            next_pop.extend(elites)

        # Fill rest with tournament + crossover + mutation
        while len(next_pop) < pop_size:
            # Select two parents
            parent1 = tournament_select(pop, stats, rng,
k=tournament_k)
            parent2 = tournament_select(pop, stats, rng,
k=tournament_k)

            # Crossover with probability
            if rng.random() < crossover_rate:
                child = crossover(parent1, parent2, rng)
            else:
                # No crossover: clone parent1
                child = parent1.copy()

            # Mutate
            child = mutate(child, rng, p_mut_gene=mutation_rate)

            next_pop.append(child)
```

```
        pop = next_pop[:pop_size]  # Ensure exact size

    return best_genome, best_stat
```

GA Flow Diagram:

text

```
Generation Loop:
│
├─ Evaluate population on S seeds → get fitness stats
├─ Sort by (feasible first, then score)
├─ Update global best
├─ Print progress
│
├─ Create next generation:
│  ├─ Copy top 10% (elitism)
│  └─ For remaining 90%:
│     ├─ Tournament select parent1
│     ├─ Tournament select parent2
│     ├─ Crossover (85% chance) → child
│     ├─ Mutate child (25% per gene)
│     └─ Add child to next generation
│
└─ Repeat
```

Part 6: Understanding Key Mechanisms

Feasibility-First Dominance

The `compare_stats` function implements a crucial concept:

python

```
def compare_stats(a: Dict[str, float], b: Dict[str, float]) ->
int:
    """
    Returns:
     1 if a is better
    -1 if b is better
     0 if equal
    """
    # Rule 1: Feasible always beats infeasible
    if a["feasible"] > b["feasible"]:  # a feasible, b infeasible
        return 1
    if b["feasible"] > a["feasible"]:  # b feasible, a infeasible
        return -1

    # Rule 2: Among same feasibility status, higher score wins
    if a["score"] > b["score"]:
        return 1
    if b["score"] > a["score"]:
        return -1

    return 0  # Tied
```

Why this matters:

- A solution with fitness=-100 but NO extinctions beats a solution with fitness=+500 but one extinction
- Forces the GA to find feasible regions first, then optimize within them
- Prevents algorithm from "gaming" the system with high-scoring but invalid solutions

Elite Re-evaluation

```
python

if reevaluate_elites:
    fresh_seeds = seed_set(base_seed=20000 + gen * 1000, S=S)
```

```
    elite_stats = [robust_eval(g, fresh_seeds, steps=steps) for
g in elites]
```

Purpose: Prevent "lucky" solutions from dominating.

Scenario without re-evaluation:

1. Solution A gets lucky on 5 seeds → fitness = 35 → becomes elite
2. Solution A copied unchanged for 20 generations
3. But Solution A was actually bad—just got lucky initial seeds!

With re-evaluation:

- Elites tested on fresh seeds each generation
- Lucky solutions eventually exposed as mediocre
- Truly robust solutions consistently perform well

Repair vs Rejection

Both ES and GA use repair (not rejection):

Repair approach (used here):

```
python
```

```
child = mutate(parent)  # Might create L=65 (out of bounds)
repair_inplace(child)   # Clamps to L=60 (MAX_L)
# Child survives with corrected values
```

Rejection approach (NOT used):

```
python
```

```
while True:
    child = mutate(parent)
    if is_valid(child):
        break  # Keep trying until valid child created
# Wasteful! Might try hundreds of times
```

Why repair is better:

- Efficient: no wasted evaluations
- Preserves genetic material: child similar to parent, just boundary-adjusted
- Essential for constrained problems like this

Part 7: Exercise 3 - Comparing ES and GA

What to Compare

After implementing both algorithms, you'll run experiments and analyze:

1. Convergence Speed:

- Which finds feasible solutions faster?
- Plot: Generation vs Best Fitness for ES and GA on same axes

2. Solution Quality:

- After N generations, which has better final fitness?
- Run validation: 20 seeds on best solution from each

3. Robustness:

- Which has higher success rate (% of seeds with no extinction)?
- Compare success_rate from validate_solution()

4. Population Dynamics:

- Examine best solutions: stable populations or oscillating?
- Plot population trajectories over 100 steps

5. Parameter Sensitivity:

- How sensitive is each algorithm to parameter settings?
- Try different μ/λ (ES) or pop_size/elite_frac (GA)

Expected Observations

ES Advantages:

- Simpler implementation (no crossover logic)
- Natural for real-valued parameters
- Self-adaptive step sizes work well
- Good for noisy fitness (median over seeds)

GA Advantages:

- Better at discrete decisions (integer populations)
- Crossover can combine complementary solutions
- Tournament selection with feasibility-first very effective
- Elitism stabilizes search

Typical Results:

- GA likely finds feasible solutions faster (discrete jumps via crossover)
- ES may find smoother, more refined solutions (continuous tuning)
- Both should converge to stable ecosystems with proper parameter tuning

Comparison Table Template

Metric	**ES (Exercise 1)**	**GA (Exercise 2)**
Generations to feasibility	___	___
Best fitness (final)	___	___
Validation success rate	__%	__%
Median min population	___	___
Population volatility	___	___
Runtime (seconds)	___	___

Part 8: Debugging Strategy

Common Issues and Fixes

Issue 1: No feasible solutions found

Symptoms: All fitness values negative, feas_rate = 0%

Causes:

- Bounds too restrictive (e.g., MAX_EAGLES too low)
- Initial populations unsustainable
- Obstacles too dense (no space for agents)

Fixes:

- Widen search bounds temporarily
- Check decode() produces valid parameters
- Run baseline with known-good parameters

Issue 2: Algorithm doesn't improve

Symptoms: Best fitness flat across generations

Causes:

- Mutation too small (no exploration)
- Mutation too large (disrupts good solutions)
- Selection pressure wrong (tournament_k)

Fixes:

```
python

# ES: Increase sigma
sigma = np.array([...]) * 0.25  # Try 25% instead of 15%

# GA: Adjust tournament size
tournament_k = 5  # Try stronger selection
```

Issue 3: RuntimeError in _random_empty_cell()

Symptoms: "Failed to sample empty cell"

Cause: Grid too crowded—not enough empty cells for initial spawn

Fix:

- Reduce total initial population (N_e0 + N_r0 + N_c0)
- Increase grid size (L)
- Decrease obstacle fraction (p_obs)

Issue 4: One species always dominates/extincts

Symptoms: Eagles always die or rabbits explode

Ecological interpretations:

- Eagles extinct: Not enough rabbits to sustain them
    - Fix: Increase N_r0 or decrease N_e0
- Rabbits extinct: Too many eagles or too few carrots
    - Fix: Decrease N_e0 or increase N_c0
- Oscillations: Classic predator-prey cycles (might be unavoidable)

Testing Strategy

Step 1: Test decode() and clamp()

```
python

# Test clamp
assert clamp(15, 10, 20) == 15
assert clamp(5, 10, 20) == 10
assert clamp(25, 10, 20) == 20

# Test decode
theta_test = np.array([45.7, 0.063, 5.2, 150.8, 450.3, 0.52,
0.48])
params = decode(theta_test)
assert 25 <= params.L <= 60
assert params.N_e0 == 5  # Should round 5.2
assert 0.40 <= params.rho_e <= 0.60
print("✓ Decode working correctly")
```

Step 2: Test fitness with known parameters

python

```
# Use baseline parameters
theta_baseline = np.array([50.0, 0.075, 6.0, 60.0, 500.0, 0.50,
0.50])
fit = fitness(theta_baseline, base_seed=1001, S=2, steps=50)
print(f"Baseline fitness: {fit}")
# Should be negative if extinction occurs, positive if stable
```

Step 3: Test operators in isolation

python

```
# Test tournament_select returns valid genome
pop_test = [random_genome(rng) for _ in range(10)]
stats_test = [{"feasible": 0.0, "score": -100} for _ in
range(10)]
stats_test[5] = {"feasible": 1.0, "score": 50}  # Make one good
selected = tournament_select(pop_test, stats_test, rng, k=10)
# With k=10, should almost always select index 5
```

Step 4: Run with minimal settings

python

```
# Test ES with tiny problem
best_theta, best_fit, history = run_es(
    generations=3,
    mu=5,
    lam=10,
    S=2,  # Only 2 seeds
    steps=20,  # Only 20 simulation steps
    verbose_every=1
)
# Should complete without errors
```

Part 9: Video Rendering - Understanding the Visualization

What the Video Shows

The `render_video()` function creates a 4-panel layout:

Panel 1: Legend

- Visual key: triangle=carrot, circle=rabbit, X=eagle, square=obstacle

Panel 2: Animation

- Main grid showing agent positions over time
- Watch movements: rabbits chasing carrots, eagles chasing rabbits
- Obstacles block paths and line-of-sight

Panel 3: HUD (Heads-Up Display)

- Current step counter
- Decoded parameters (L, p_obs, population sizes, ratios)
- Live population counts

Panel 4: Progress Bar

- Visual indicator of simulation progress (0-100 steps)

What to Look For in Videos

Signs of a good solution:

- All three species visible throughout (no mass die-offs)
- Populations fluctuate but stay bounded
- Spatial distribution: not all clumped in one corner
- Dynamic interactions: eagles hunting, rabbits fleeing/feeding

Signs of problems:

- Rapid population collapse (fitness landscape too harsh)
- Static agents (movement parameters wrong)
- Overcrowding in small region (need larger L or fewer agents)
- One species vanishes early (ecological imbalance)

Frame Capture Process

python

```python
def simulate_frames(theta_real, seed, steps):
    # Setup simulation with decoded parameters
    params = decode(theta_real)
    env = GridWorld(...)

    frames = []
    for t in range(steps):
        # Capture positions BEFORE stepping
        carrots = np.array([[y, x] for (x,y) in env.carrots])
        rabbits = np.array([[a.y, a.x] for a in env.rabbits])
        eagles = np.array([[a.y, a.x] for a in env.eagles])

        frames.append({
            "t": t,
            "carrots": carrots,
            "rabbits": rabbits,
            "eagles": eagles,
            ...
        })

        env.step()  # Update simulation

    return params, env.L, obstacles, frames
```

Critical insight: Positions captured before each step, so frame 0 shows initial conditions.

Part 10: Parameter Tuning Guide

ES Parameter Recommendations

Parameter	Recommended	Reasoning
generations	60-80	Enough for convergence
μ (mu)	10-15	Parent pool size
λ (lambda)	40-60	Offspring pool (λ > 4μ typical)
S (seeds)	5-7	Balance robustness vs computation
sigma_init	15% of range	Moderate initial exploration

Tuning tips:

- If stuck in local optimum: Increase sigma or λ
- If too random: Decrease sigma or increase μ
- If evaluation slow: Reduce S or steps

GA Parameter Recommendations

Parameter	Recommended	Reasoning
generations	40-60	GA converges faster than ES

pop_size	60-100	Maintain diversity
elite_frac	0.05-0.15	Preserve best 5-15%
tournament_k	3-5	Moderate selection pressure
crossover_rate	0.70-0.90	Crossover is primary operator
mutation_rate	0.15-0.30	Per-gene probability

Tuning tips:

- If converges too fast (premature): Increase pop_size, decrease elite_frac
- If too slow: Increase tournament_k, decrease pop_size
- If stuck: Increase mutation_rate

Simulation Parameter Bounds

Exercise 1 (ES):

- L: – Larger grids for continuous search
- N_e0: – Predator scarcity
- N_r0: – Moderate prey
- N_c0: – Ample resources

Exercise 2 (GA):

- L: – Smaller grids test GA's discrete handling
- N_e0: – Even fewer predators
- N_r0: – More prey
- N_c0: – Tighter resource constraint

Why different bounds?: Tests each algorithm's strengths. GA should handle integer constraints better; ES should handle continuous ratios better.

Part 11: Advanced Understanding - Why This Problem Is Hard

Fitness Landscape Characteristics

1. High-Dimensional (7 parameters)

- Exponentially large search space
- Can't visualize or enumerate
- Need intelligent search (EA)

2. Multi-Modal (many local optima)

- Multiple parameter combinations work
- Example: [L=30, N_r0=50, N_c0=400] vs [L=50, N_r0=80, N_c0=500] both feasible
- Algorithms must avoid getting stuck

3. Deceptive

- Parameters interact non-linearly
- Example: Increasing N_e0 might improve fitness up to a point, then collapse ecosystem
- Gradient-free methods essential

4. Noisy

- Stochastic simulation→different results each run
- Median aggregation over seeds combats this
- But adds computational cost (S × evaluations)

5. Constrained (hard feasibility boundary)

- Most of search space is infeasible (extinction occurs)
- Feasible region might be small, disconnected pockets
- Feasibility-first dominance crucial for finding viable regions

Ecological Balance Dynamics

Trophic Cascade:

```
text

Carrots → Rabbits → Eagles
```

```
If N_c0 ↓ (fewer carrots):
  → Rabbits starve
  → Eagle food supply ↓
  → Eagles starve
  → ALL GO EXTINCT

If N_e0 ↑ (more eagles):
  → Rabbits hunted aggressively
  → Rabbits ↓
  → Eagles starve (no prey)
  → PREDATOR-PREY COLLAPSE
```

Optimal solutions balance:

- Enough carrots to sustain rabbits
- Enough rabbits to sustain eagles (but not so many eagles that rabbits can't reproduce)
- Proper sex ratios (need males AND females)
- Spatial structure (obstacles create refuges for rabbits)

Parameter Interactions

L × p_obs interaction:

- Large L + high p_obs = many obstacles, fragmented space
- Agents can't find mates/food
- Small L + low p_obs = open arena, no hiding
- Eagles dominate

N_e0 × ρ_e interaction:

- N_e0=4, ρ_e=0.20 → 1 female, 3 males
- Reproduction bottlenecked by single female
- N_e0=4, ρ_e=0.50 → 2 females, 2 males
- Balanced reproduction

These interactions make gradient-based optimization impossible!

Part 12: Conceptual Summary & Checklist

Core Concepts You Now Understand

✅ Ecosystem Simulation: Multi-agent predator-prey-resource dynamics with spatial structure and stochasticity

✅ Optimization Problem: Finding stable initial conditions in a high-dimensional, constrained, noisy search space

✅ Evolution Strategy (ES): Mutation-driven, real-valued, (μ,λ) selection, continuous optimization

✅ Genetic Algorithm (GA): Crossover-driven, mixed-type, tournament selection, discrete optimization

✅ Robustness via median: Aggregating over multiple seeds handles simulation noise

✅ Feasibility-first dominance: Prioritizing constraint satisfaction before objective optimization

✅ Repair mechanisms: Clamping and rounding to maintain parameter validity

✅ Encoding/Decoding: Transforming real-valued search vectors into typed simulation parameters

Implementation Checklist

Exercise 1 (ES):

- Implement `clamp(x, lo, hi)`
- Implement `decode(theta_real)` with clamping and rounding
- Implement `fitness(theta_real, ...)` with median aggregation
- Implement `run_es(...)` with (μ, λ) selection loop
- Run ES and observe convergence logs
- Validate best solution on 20 seeds
- Render video of best solution

Exercise 2 (GA):

- Implement `tournament_select(...)` with feasibility-first comparison
- Implement `mutate(...)` with type-aware strategies

- Implement `crossover(...)` with uniform swap + BLX-α
- Implement `run_ga(...)` with elitism and generational loop
- Run GA and observe convergence logs
- Validate best solution on 20 seeds
- Render video of best solution

Exercise 3 (Comparison):

- Run both ES and GA with comparable computational budgets
- Compare convergence speed (generations to feasibility)
- Compare final solution quality (validation fitness)
- Compare robustness (success rate across seeds)
- Analyze population trajectories in videos
- Document insights and trade-offs

Success Criteria

Minimum viable solution:

- Feasibility rate > 0% (at least some seeds survive)
- Best fitness > 0 (better than penalty baseline)
- Code runs without errors

Good solution:

- Feasibility rate > 50%
- Best fitness > 20
- Visible stability in rendered video

Excellent solution:

- Feasibility rate > 80%
- Best fitness > 40
- Smooth population dynamics, minimal oscillations
- Fast convergence (< 30 generations to good solutions)